

Apostila CS50x — Introdução à Ciência da Computação

Baseada no curso CS50x de Harvard (David J. Malan) — Semanas 0 a 10

Compilado a partir de cs50.harvard.edu/x/weeks/

Agosto de 2026

Sumário

Introdução	4
Semana 0 — Scratch	5
0.1 Visão geral	5
0.2 Resolução de problemas: entrada, algoritmo, saída	5
0.3 Representação de dados	5
Unário, decimal e binário	6
ASCII e Unicode	6
Cores (RGB)	6
0.4 Abstração	6
0.5 Scratch: os blocos de construção da programação	7
0.6 Exercícios — Semana 0	7
Semana 1 — C	9
1.1 Visão geral	9
1.2 Do código-fonte ao código de máquina	9
1.3 Estrutura básica de um programa em C	9
1.4 Tipos, variáveis e operadores	10
Overflow de inteiros	10
Imprecisão de ponto flutuante	10
1.5 Condicionais	11
1.6 Loops	11
1.7 Constantes, comentários e boas práticas	11
1.8 CLI, GUI e Linux	12
1.9 Exercícios — Semana 1	12
Semana 2 — Arrays	13
2.1 Visão geral	13
2.2 As quatro etapas da compilação	13
2.3 Depuração (debugging)	13
2.4 Arrays	13
2.5 Strings como arrays de caracteres	14
2.6 Argumentos de linha de comando	14
2.7 Criptografia (Cifra de César)	15
2.8 Exercícios — Semana 2	15
Semana 3 — Algoritmos	16

3.1 Visão geral	16
3.2 Algoritmos de busca	16
Busca linear — $O(n)$	16
Busca binária — $O(\log n)$	16
3.3 Notação assintótica (Big O)	17
3.4 Algoritmos de ordenação	17
Bubble Sort — $O(n^2)$	17
Selection Sort — $O(n^2)$	17
Merge Sort — $O(n \log n)$	18
3.5 Recursão	18
3.6 Exercícios — Semana 3	18
Semana 4 — Memória	20
4.1 Visão geral	20
4.2 Ponteiros	20
4.3 Strings revisitadas: string é char *	20
4.4 Pilha (Stack) e Heap	20
4.5 Alocação dinâmica de memória	21
4.6 Falhas de segmentação e buffer overflow	21
4.7 Entrada/saída de arquivos (File I/O)	21
4.8 Imagens (formato BMP)	22
4.9 Exercícios — Semana 4	22
Semana 5 — Data Structures (Estruturas de Dados)	23
5.1 Visão geral	23
5.2 Tipos Abstratos de Dados (TADs)	23
5.3 Filas (Queues) e Pilhas (Stacks)	23
5.4 Listas ligadas (Linked Lists)	23
5.5 Árvores e Árvores Binárias de Busca (BST)	24
5.6 Tabelas Hash (Hash Tables)	24
5.7 Tries	24
5.8 Exercícios — Semana 5	24
Semana 6 — Python	26
6.1 Visão geral	26
6.2 De C para Python: o que muda	26
6.3 Variáveis e tipos	26
6.4 Condicionais e expressões booleanas	27
6.5 Loops	27
6.6 Funções, argumentos e valores de retorno	27
6.7 Módulos e pacotes	28
6.8 Exercícios — Semana 6	28
Semana 7 — SQL	29
7.1 Visão geral	29
7.2 Bancos de dados relacionais e tabelas	29
7.3 Tipos de dados em SQL	29
7.4 As instruções fundamentais (CRUD)	29

7.5 Restrições e índices	30
7.6 Funções e palavras-chave úteis	30
7.7 Transações e condições de corrida	30
7.8 Ataques de SQL Injection	31
7.9 Exercícios — Semana 7	31
Semana 8 — HTML, CSS, JavaScript	32
8.1 Visão geral	32
8.2 Como funciona a Internet	32
8.3 HTTP: como o navegador conversa com o servidor	32
8.4 HTML: estrutura da página	32
8.5 CSS: estilizando a página	33
8.6 Expressões regulares (Regex)	33
8.7 JavaScript: interatividade no navegador	34
8.8 Exercícios — Semana 8	34
Semana 9 — Flask	35
9.1 Visão geral	35
9.2 Estrutura mínima de uma aplicação Flask	35
9.3 Rotas com múltiplos métodos e formulários	35
9.4 Sessões e cookies	36
9.5 Exercícios — Semana 9	36
Semana 10 — The End	38
10.1 Visão geral	38
10.2 O Projeto Final	38
10.3 Recapitulando a jornada	38
10.4 Seminários e além do CS50x	39
10.5 Exercícios — Semana 10	39
Referências	40

Introdução

Esta apostila resume o conteúdo do **CS50x — Introdução à Ciência da Computação de Harvard**, ministrado por David J. Malan, cobrindo as **Semanas 0 a 10** conforme organizado em cs50.harvard.edu/x/weeks/. Cada semana traz: os conceitos centrais, explicações aprofundadas, exemplos de código comentados e exercícios de fixação.

A trilha de linguagens do curso é: **Scratch → C → Python → SQL → HTML/CSS/JavaScript → Flask (Python)**. A cada semana o nível de abstração sobe: primeiro você aprende a *pensar* como um cientista da computação (Scratch), depois entende o que acontece “por baixo do capô” com C (memória, ponteiros, compilação), e só então parte para linguagens de alto nível como Python e para a Web.

Semana 0 — Scratch

0.1 Visão geral

A Semana 0 introduz o **pensamento computacional**: a habilidade de resolver problemas dividindo-os em passos precisos e inequívocos (algoritmos), independente da linguagem de programação usada depois. Os tópicos oficiais são:

Ciência da Computação. Pensamento Computacional. Resolução de Problemas: Entradas, Saídas. Representação: Unário, Binário, Decimal, ASCII, Unicode, RGB. Abstração. Algoritmos. Tempo de Execução. Pseudocódigo. Scratch: Funções, Argumentos, Valores de Retorno; Variáveis; Expressões Booleanas, Condicionais; Loops; Eventos; Threads.

0.2 Resolução de problemas: entrada, algoritmo, saída

Todo problema computacional pode ser modelado como:

Entrada (Input) → Algoritmo (processamento) → Saída (Output)

Um **algoritmo** é uma sequência finita e precisa de passos para resolver um problema. O mesmo problema pode ter vários algoritmos diferentes, com eficiências diferentes — é aqui que nasce a ideia de **tempo de execução (running time)**, formalizada mais tarde com a notação Big O (Semana 3).

Exemplo clássico — buscar um nome em uma lista telefônica:

- *Busca linear*: olhar página por página, uma de cada vez.
- *Busca binária*: abrir no meio, decidir para qual metade ir, e repetir — dividindo o problema pela metade a cada passo.

A busca binária é exponencialmente mais rápida à medida que a lista cresce — esse é o tipo de intuição que a Semana 0 quer construir antes mesmo de escrever código.

0.3 Representação de dados

Computadores só entendem **eletricidade ligada/desligada**, por isso tudo é representado em **binário** (base 2).

Unário, decimal e binário

- **Unário:** cada dígito vale 1 (ex.: contar nos dedos). Representar 5 exige 5 símbolos: 11111.
- **Decimal (base 10):** o sistema humano, com dígitos de 0 a 9. O número 123 vale $1 \times 10^2 + 2 \times 10^1 + 3 \times 10^0$.
- **Binário (base 2):** só existem os dígitos 0 e 1 (*bits*). O número 1011_2 vale:

$$1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 8 + 0 + 2 + 1 = 11_{10}$$

Exercício resolvido: converta 10110_2 para decimal.

$$1 \times 16 + 0 \times 8 + 1 \times 4 + 1 \times 2 + 0 \times 1 = 16 + 4 + 2 = 22_{10}$$

ASCII e Unicode

- **ASCII** representa caracteres usando números de 0 a 127 (7 bits), ex.: A = 65, a = 97, 0 = 48.
- **Unicode** estende essa ideia para cobrir praticamente todos os idiomas e símbolos do mundo (incluindo emojis 🍌), usando mais bits por caractere (UTF-8, UTF-16, etc.).

Cores (RGB)

Uma cor de tela é representada por três números de 0 a 255, indicando a intensidade de **vermelho (Red)**, **verde (Green)** e **azul (Blue)**. Por exemplo, (255, 0, 0) é vermelho puro, e (0, 0, 0) é preto.

0.4 Abstração

Abstração é o ato de esconder detalhes complexos por trás de uma interface simples. Um algoritmo escrito em **pseudocódigo** (passos em linguagem natural, sem se prender à sintaxe de uma linguagem específica) é uma forma de abstração — ele descreve *o quê* fazer, sem se preocupar com *como* uma linguagem específica implementa isso.

Exemplo de pseudocódigo para buscar um nome numa lista telefônica ordenada (busca binária):

Se a lista está vazia:

Retornar "não encontrado"

Senão:

Escolher a página do meio

Se o nome está nessa página:

Parar

Senão se o nome vem antes:

Buscar na metade esquerda

Senão:

Buscar na metade direita

0.5 Scratch: os blocos de construção da programação

Scratch é uma linguagem visual (arrastar e soltar blocos) criada pelo MIT Media Lab, usada para introduzir conceitos de programação sem exigir domínio de sintaxe textual. Os conceitos aprendidos aqui reaparecem em **toda** linguagem de programação:

Conceito	O que significa	Analogia em Scratch
Variáveis	Um “espaço na memória” com nome, que guarda um valor que pode mudar	Bloco laranja minha variável
Expressões booleanas	Expressões que resultam em verdadeiro/falso	Bloco hexagonal <...>
Condicionais	Executar código somente se uma condição for verdadeira	Blocos se ... então / se ... senão
Loops	Repetir um bloco de instruções	repita, sempre, repita até
Funções (blocos personalizados)	Agrupar instruções reutilizáveis, com nome	Meu Bloco
Argumentos	Valores passados para dentro de uma função	Entradas de um “Meu Bloco”
Valores de retorno	O resultado que uma função devolve	Reporters (blocos ovais que “devolvem” um valor)
Eventos	Código que roda em resposta a algo (clique, tecla, mensagem)	quando bandeira verde for clicada, quando tecla espaço pressionada
Threads	Múltiplos scripts rodando “ao mesmo tempo”	Vários scripts anexados a diferentes blocos de evento

Exemplo em pseudocódigo Scratch — um sprite que anda para frente e volta ao bater na borda:

```
quando bandeira verde for clicada
sempre
  mover 10 passos
  se está tocando na borda então
    virar 180 graus
```

0.6 Exercícios — Semana 0

1. Converta o número binário 1101_2 para decimal.
2. Converta o número decimal 37 para binário.
3. Explique, com suas palavras, a diferença entre um algoritmo de busca linear e um de busca binária. Em que situação a busca binária **não** funciona?
4. Escreva em pseudocódigo um algoritmo para encontrar o maior número em uma lista de números.

5. Dê um exemplo do dia a dia (fora da computação) de uso de **abstração** — algo cujos detalhes internos você ignora ao usá-lo.
6. Em Scratch, qual a diferença entre um bloco sempre e um bloco repita 10?
7. Explique a diferença entre uma **variável** e um **valor de retorno** de uma função.

Semana 1 — C

1.1 Visão geral

A partir daqui o curso migra para **C**, uma linguagem compilada de baixo nível que expõe explicitamente conceitos que ficavam escondidos no Scratch (tipos, memória, compilação). Tópicos oficiais:

C. Código-fonte. Código de máquina. Compilador. Correção, Design, Estilo. Visual Studio Code. Realce de sintaxe. Sequências de escape. Arquivos de cabeçalho. Bibliotecas. Manual Pages. Tipos. Condicionais. Variáveis. Loops. Linux. GUI. CLI. Constantes. Comentários. Pseudocódigo. Operadores. Overflow de inteiros. Imprecisão de ponto flutuante.

1.2 Do código-fonte ao código de máquina

- **Código-fonte:** o texto que você escreve, em C (.c), legível por humanos.
- **Compilador:** um programa (clang, gcc) que traduz código-fonte em **código de máquina** (binário, 0s e 1s), o único formato que o processador realmente executa.
- O processo de compilação tem 4 etapas: **pré-processamento** → **compilação** → **montagem (assembling)** → **ligação (linking)** — detalhado na Semana 2.

```
# compilar hello.c em um executável chamado "hello"
make hello
./hello
```

O make automatiza a chamada ao compilador (clang -o hello hello.c -lcs50 etc.), verificando erros de sintaxe antes de gerar o binário.

1.3 Estrutura básica de um programa em C

```
#include <cs50.h>
#include <stdio.h>

int main(void)
{
    string name = get_string("What's your name? ");
```

```
printf("hello, %s\n", name);
}
```

- `#include` importa **bibliotecas** (blocos de código já prontos). `cs50.h` traz funções como `get_string`; `stdio.h` traz `printf`.
- `int main(void)` é a função principal: todo programa em C começa a executar aqui.
- `printf` usa **especificadores de formato** (`%s` para string, `%i` para inteiro, `%f` para float, `%c` para char) para inserir valores dentro do texto.
- `\n` é uma **sequência de escape** que representa uma quebra de linha.

1.4 Tipos, variáveis e operadores

C exige que você declare o **tipo** de cada variável (linguagem *estaticamente tipada*):

Tipo	Descrição	Exemplo
<code>int</code>	Número inteiro (~4 bytes)	<code>int idade = 25;</code>
<code>long</code>	Inteiro maior (~8 bytes)	<code>long populacao = 8000000000;</code>
<code>float</code>	Ponto flutuante (~4 bytes, menos preciso)	<code>float pi = 3.14;</code>
<code>double</code>	Ponto flutuante de precisão dupla (~8 bytes)	<code>double pi = 3.14159265;</code>
<code>char</code>	Um único caractere	<code>char letra = 'A';</code>
<code>bool</code>	Verdadeiro/falso (via <code>cs50.h</code>)	<code>bool ligado = true;</code>
<code>string</code>	Cadeia de caracteres (via <code>cs50.h</code>)	<code>string nome = "Ana";</code>

Operadores aritméticos (+ - * / %), relacionais (== != < > <= >=) e lógicos (&& || !) funcionam como em outras linguagens.

Overflow de inteiros

Como cada tipo tem um número fixo de bits, existe um valor máximo representável. Se você ultrapassar esse limite, o valor “estoura” (*overflow*) e volta para o começo — como um odômetro de carro que volta a zero.

```
// int de 32 bits tem no máximo 2147483647
int x = 2147483647;
x = x + 1; // comportamento indefinido / vira número negativo
```

Imprecisão de ponto flutuante

Nem todo número decimal tem representação binária exata (assim como $1/3$ não tem representação decimal exata). Por isso, cálculos com `float`/`double` podem gerar

pequenos erros de arredondamento.

```
float x = 1.0 / 3.0;    // 0.333333... – nunca é exato
```

1.5 Condicionais

```
int x = get_int("x: ");
int y = get_int("y: ");

if (x < y)
{
    printf("x is less than y\n");
}
else if (x > y)
{
    printf("x is greater than y\n");
}
else
{
    printf("x is equal to y\n");
}
```

1.6 Loops

```
// while: repete enquanto a condição for verdadeira
int i = 0;
while (i < 3)
{
    printf("meow\n");
    i++;
}

// for: mais compacto para contagens
for (int i = 0; i < 3; i++)
{
    printf("meow\n");
}
```

1.7 Constantes, comentários e boas práticas

```
const int TAMANHO_MAXIMO = 100;    // constante: não pode ser alterada depois

// Isto é um comentário de uma linha
```

```
/* Isto é um  
   comentário de várias linhas */
```

O curso avalia todo pset (problem set) em três eixos, cobrados pelo check50/style50:

1. **Correção (correctness):** o programa faz o que deveria?
2. **Design:** a lógica é eficiente e bem estruturada?
3. **Estilo (style):** o código segue convenções de formatação e nomenclatura (indentação, espaçamento, nomes claros)?

1.8 CLI, GUI e Linux

O curso usa um ambiente de desenvolvimento em nuvem baseado em **Linux** (via VS Code / cs50.dev). Diferente de uma **GUI** (interface gráfica, com janelas e cliques), você interage por uma **CLI** (interface de linha de comando, digitando comandos):

```
cd pset1          # mudar de diretório  
ls                # listar arquivos  
mkdir nova_pasta  # criar pasta  
rm arquivo.c      # remover arquivo  
code hello.c      # abrir arquivo no editor
```

1.9 Exercícios — Semana 1

1. Escreva um programa em C que peça o nome do usuário com `get_string` e imprima "Hello, <nome>!".
2. Qual a diferença entre `int` e `long`? E entre `float` e `double`?
3. O que é *overflow* de inteiro? Dê um exemplo numérico com um tipo de 8 bits (valor máximo 255).
4. Escreva um `for` que imprima os números de 1 a 10.
5. Reescreva o loop do exercício anterior usando `while` em vez de `for`.
6. Por que `0.1 + 0.2` pode não resultar exatamente em `0.3` em ponto flutuante?
7. O que faz o comando `make nome_do_programa` no terminal do CS50?

Semana 2 — Arrays

2.1 Visão geral

Pré-processamento. Compilação. Montagem (Assembling). Ligação (Linking). Depuração (Debugging). Arrays. Strings. Argumentos de linha de comando. Criptografia.

2.2 As quatro etapas da compilação

1. **Pré-processamento:** resolve as linhas `#include`, colando o conteúdo das bibliotecas.
2. **Compilação:** traduz C para linguagem assembly (uma representação intermediária, ainda legível).
3. **Montagem (assembling):** converte assembly em código de máquina (binário, `.o`).
4. **Ligação (linking):** combina seu código de máquina com o das bibliotecas usadas, gerando o executável final.

2.3 Depuração (debugging)

Depurar é o processo de encontrar e corrigir erros (*bugs*). Técnicas ensinadas no CS50:

- Usar `printf` para inspecionar valores de variáveis em pontos-chave do programa.
- Usar o **debugger** integrado do VS Code (pontos de parada / *breakpoints*, executar linha a linha).
- Ler mensagens de erro do compilador com atenção — o número da linha indicado nem sempre é exatamente onde está o erro.

2.4 Arrays

Um **array** é um bloco contíguo de memória que guarda vários valores do **mesmo tipo**, acessados por índice (começando em 0).

```
int scores[3];  
scores[0] = 72;
```

```

scores[1] = 73;
scores[2] = 33;

// ou, de forma compacta:
int scores[] = {72, 73, 33};

for (int i = 0; i < 3; i++)
{
    printf("%i\n", scores[i]);
}

```

2.5 Strings como arrays de caracteres

Em C, uma string é, na verdade, um **array de char** terminado por um caractere especial `\0` (*null terminator*), que marca o fim do texto.

```

string s = "HI!";
// na memória: s[0]='H', s[1]='I', s[2]='!', s[3]='\0'

printf("%c\n", s[0]); // imprime H

for (int i = 0; i < strlen(s); i++)
{
    printf("%c\n", s[i]);
}

```

A função `strlen` (de `string.h`) percorre o array até encontrar o `\0` para calcular o tamanho.

2.6 Argumentos de linha de comando

O próprio `main` pode receber argumentos digitados no terminal:

```

int main(int argc, string argv[])
{
    // argc = quantidade de argumentos (incluindo o nome do programa)
    // argv = array de strings com os argumentos
    if (argc == 2)
    {
        printf("hello, %s\n", argv[1]);
    }
    else
    {
        printf("hello, world\n");
    }
}

```

Executando `./hello Ana` no terminal, `argv[0]` é `./hello` e `argv[1]` é `Ana`.

2.7 Criptografia (Cifra de César)

Uma introdução prática a criptografia é a **Cifra de César**, que desloca cada letra do alfabeto em k posições.

```
// pseudocódigo da cifra de César
para cada letra c no texto:
    se c é uma letra:
        nova_letra = ((c - 'a') + chave) % 26 + 'a'
    senão:
        nova_letra = c // mantém espaços/pontuação sem alterar
```

Esse é o mesmo raciocínio usado no pset2 (Substitution / Caesar), que reforça manipulação de strings e arrays.

2.8 Exercícios — Semana 2

1. Declare um array de 5 inteiros e preencha-o com um for com os quadrados de 1 a 5 (1, 4, 9, 16, 25).
2. Explique por que `strlen("cs50")` retorna 4 mesmo o array tendo, na memória, 5 posições.
3. Escreva uma função que receba uma string e conte quantas vogais ela contém.
4. O que aconteceria se você tentasse acessar `scores[10]` de um array declarado com apenas 3 elementos?
5. Escreva o pseudocódigo (ou código em C) da Cifra de César para descriptografar um texto (operação inversa).
6. Quais são as 4 etapas do processo de compilação, na ordem?
7. Rodando `./programa a b c`, quanto vale `argc`?

Semana 3 — Algoritmos

3.1 Visão geral

Busca: Busca Linear, Busca Binária. Ordenação: Bubble Sort, Selection Sort, Merge Sort. Notação Assintótica: O , Ω , Θ . Recursão.

3.2 Algoritmos de busca

Busca linear — $O(n)$

Percorre o array do início ao fim, um elemento de cada vez.

```
bool busca_linear(int valor, int array[], int n)
{
    for (int i = 0; i < n; i++)
    {
        if (array[i] == valor)
        {
            return true;
        }
    }
    return false;
}
```

Busca binária — $O(\log n)$

Exige um array **ordenado**. Compara o elemento do meio; se não for o buscado, descarta metade do array e repete.

```
bool busca_binaria(int valor, int array[], int inicio, int fim)
{
    if (inicio > fim)
    {
        return false;
    }
    int meio = (inicio + fim) / 2;
    if (array[meio] == valor)
    {

```

```

        return true;
    }
    else if (array[meio] < valor)
    {
        return busca_binaria(valor, array, meio + 1, fim);
    }
    else
    {
        return busca_binaria(valor, array, inicio, meio - 1);
    }
}

```

3.3 Notação assintótica (Big O)

Mede como o **tempo de execução** de um algoritmo cresce conforme o tamanho da entrada (n) aumenta, ignorando constantes e detalhes de hardware:

Notação	Significado	Leitura
$O(g(n))$	Limite superior (pior caso)	“no máximo, cresce como...”
$\Omega(g(n))$	Limite inferior (melhor caso)	“no mínimo, cresce como...”
$\Theta(g(n))$	Quando O e Ω coincidem	“cresce exatamente como...”

Complexidades comuns, da mais rápida para a mais lenta: $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2)$.

3.4 Algoritmos de ordenação

Bubble Sort — $O(n^2)$

Compara pares adjacentes e os troca de posição se estiverem fora de ordem, repetindo até que nada mais precise ser trocado.

```

para i de 0 até n-2:
    para j de 0 até n-2-i:
        se array[j] > array[j+1]:
            trocar(array[j], array[j+1])

```

Selection Sort — $O(n^2)$

A cada passo, encontra o **menor** elemento restante e o coloca na posição correta.

```

para i de 0 até n-1:
    menor = i
    para j de i+1 até n-1:
        se array[j] < array[menor]:
            menor = j
    trocar(array[i], array[menor])

```

Merge Sort — $O(n \log n)$

Estratégia **dividir para conquistar** (divide and conquer): divide o array ao meio recursivamente até sobraem partes de 1 elemento (já “ordenadas”), depois **intercala (merge)** as partes de volta, em ordem.

```

merge_sort(array):
    se o array tem 1 elemento ou menos:
        retornar (já está ordenado)
    dividir o array em metade esquerda e metade direita
    merge_sort(metade esquerda)
    merge_sort(metade direita)
    intercalar as duas metades ordenadas em um único array

```

O Merge Sort é mais rápido que Bubble/Selection Sort para arrays grandes, ao custo de usar memória extra para armazenar as metades temporárias.

3.5 Recursão

Uma função **recursiva** chama a si mesma, sempre progredindo em direção a um **caso-base** (condição de parada), evitando recursão infinita.

```

int fatorial(int n)
{
    if (n <= 1)           // caso-base
    {
        return 1;
    }
    return n * fatorial(n - 1); // caso recursivo
}

```

$fatorial(4) = 4 * fatorial(3) = 4 * 3 * fatorial(2) = \dots = 4 * 3 * 2 * 1 = 24$.

3.6 Exercícios — Semana 3

1. Qual a complexidade (Big O) da busca linear no pior caso? E da busca binária?
2. Por que a busca binária exige que o array esteja ordenado?
3. Ordene manualmente o array [5, 2, 8, 1] usando Selection Sort, mostrando cada passo.
4. Explique a diferença entre O , Ω e Θ com suas palavras.

5. Escreva uma função recursiva em C que calcule a soma dos números de 1 até n.
6. Por que o Merge Sort é considerado mais eficiente que o Bubble Sort para arrays grandes?
7. O que aconteceria se uma função recursiva não tivesse um caso-base?

Semana 4 — Memória

4.1 Visão geral

Ponteiros. Falhas de Segmentação (Segmentation Faults). Alocação Dinâmica de Memória. Pilha (Stack). Heap. Buffer Overflow. Entrada/Saída de Arquivos (File I/O). Imagens.

4.2 Ponteiros

Toda variável mora em algum **endereço de memória**. Um **ponteiro** é uma variável que guarda o *endereço* de outra variável, em vez do valor em si.

```
int n = 50;
int *p = &n;      // p guarda o ENDEREÇO de n (& = "endereço de")

printf("%p\n", p);    // imprime o endereço
printf("%i\n", *p);   // * = "dereferenciar": acessa o VALOR no endereço -> 50
```

Ponteiros e arrays estão intimamente ligados: o nome de um array, em C, decai para um ponteiro para seu primeiro elemento.

```
int array[] = {10, 20, 30};
int *p = array;      // p aponta para array[0]
printf("%i\n", *(p+1)); // acessa array[1] -> 20
```

4.3 Strings revisitadas: string é char *

Na prática, string (de cs50.h) é um “apelido” para char * — um ponteiro para o primeiro caractere de um array de char. Isso explica por que copiar uma string ingenuamente (string s2 = s1;) apenas copia o *endereço*, não o conteúdo — as duas variáveis passam a apontar para o mesmo texto na memória.

4.4 Pilha (Stack) e Heap

A memória de um programa é organizada em regiões:

- **Stack (pilha):** guarda variáveis locais e informações de chamadas de função. Cresce “para baixo” a cada chamada de função e encolhe quando a função retorna.
- **Heap:** área de memória alocada manualmente em tempo de execução (com malloc), útil quando você não sabe de antemão quanto espaço vai precisar. Cresce “para cima”.

Se o Stack e o Heap “colidirem” (memória esgotada), ocorre um **estouro de pilha (stack overflow)** — comum em recursão infinita.

4.5 Alocação dinâmica de memória

```
#include <stdlib.h>

int *lista = malloc(5 * sizeof(int)); // aloca espaço para 5 inteiros no heap
if (lista == NULL)
{
    // malloc pode falhar (memória insuficiente) – sempre verificar!
    return 1;
}

lista[0] = 42;
// ... usar lista ...

free(lista); // devolver a memória ao sistema quando não precisar mais
```

Esquecer o free causa **vazamento de memória (memory leak)** — memória “perdida” que o programa não consegue mais usar nem devolver.

4.6 Falhas de segmentação e buffer overflow

- **Segmentation fault:** ocorre ao tentar acessar memória que não pertence ao seu programa (ex.: dereferenciar um ponteiro NULL, ou acessar um índice de array fora dos limites).
- **Buffer overflow:** escrever além dos limites de um buffer (array) alocado, sobrescrevendo memória adjacente — uma das falhas de segurança mais exploradas historicamente.

```
char buffer[3];
strcpy(buffer, "AAAA"); // escreve 5 bytes ("AAAA\0") em um buffer de 3! Perigoso.
```

4.7 Entrada/saída de arquivos (File I/O)

```
#include <stdio.h>
```

```
FILE *arquivo = fopen("dados.txt", "w"); // abrir para escrita
if (arquivo != NULL)
{
    fprintf(arquivo, "Olá, mundo!\n");
    fclose(arquivo); // sempre fechar o arquivo!
}
```

4.8 Imagens (formato BMP)

O pset4 (Filter/Recover) trabalha com arquivos **BMP**, onde a imagem é representada como uma matriz de pixels, cada um com valores RGB. Manipular imagens em C é uma aplicação direta de arrays 2D, structs (para representar um RGBTRIPLE) e ponteiros, servindo de ponte para a Semana 5 (Data Structures).

4.9 Exercícios — Semana 4

1. O que faz o operador & em C? E o operador *?
2. Escreva código que declare um `int x = 10;` e um ponteiro `p` que aponte para `x`, depois imprima o valor de `x` **através** do ponteiro.
3. Qual a diferença entre Stack e Heap?
4. Por que é importante chamar `free()` depois de usar `malloc()`?
5. O que é um *segmentation fault*? Dê um exemplo de código que causaria um.
6. Explique com suas palavras o que é um *buffer overflow* e por que ele é um risco de segurança.
7. Por que sempre verificar se `malloc()` (ou `fopen()`) retornou `NULL`?

Semana 5 — Data Structures (Estruturas de Dados)

5.1 Visão geral

Tipos Abstratos de Dados. Filas, Pilhas. Listas Ligadas. Árvores, Árvores Binárias de Busca. Tabelas Hash. Tries.

5.2 Tipos Abstratos de Dados (TADs)

Um **TAD** define *o quê* uma estrutura de dados faz (sua interface/comportamento) sem especificar *como* ela é implementada internamente — outra aplicação do conceito de abstração da Semana 0.

5.3 Filas (Queues) e Pilhas (Stacks)

- **Fila (Queue):** FIFO — *First In, First Out* (o primeiro que entra é o primeiro que sai). Como uma fila de banco. Operações: enqueue (entrar), dequeue (sair).
- **Pilha (Stack):** LIFO — *Last In, First Out* (o último que entra é o primeiro que sai). Como uma pilha de pratos. Operações: push (empilhar), pop (desempilhar).

5.4 Listas ligadas (Linked Lists)

Diferente de um array (tamanho fixo, contíguo na memória), uma **lista ligada** é uma sequência de **nós**, cada um contendo um valor e um ponteiro para o próximo nó — permitindo crescer e encolher dinamicamente.

```
typedef struct node
{
    int number;
    struct node *next;
}
node;

// criando um nó dinamicamente
node *n = malloc(sizeof(node));
```



```
n->number = 42;
n->next = NULL;
```

Vantagem: inserção/remoção são $O(1)$ (com o ponteiro certo em mãos). **Desvantagem:** acesso a um elemento no meio exige percorrer a lista a partir do início — $O(n)$, ao contrário de arrays, que têm acesso direto $O(1)$ por índice.

5.5 Árvores e Árvores Binárias de Busca (BST)

Uma **árvore** é uma estrutura hierárquica de nós conectados. Uma **Árvore Binária de Busca (BST)** mantém, para cada nó, todos os valores menores à esquerda e todos os maiores à direita — permitindo busca em $O(\log n)$ (se a árvore estiver balanceada), combinando a velocidade da busca binária com a flexibilidade de crescer dinamicamente.

```
typedef struct node
{
    int number;
    struct node *left;
    struct node *right;
}
node;
```

5.6 Tabelas Hash (Hash Tables)

Uma **tabela hash** combina arrays e listas ligadas: uma **função hash** converte uma chave (ex.: uma palavra) em um índice de array; cada posição do array (“bucket”) guarda uma lista ligada dos itens que caíram naquele índice (para tratar colisões). Na prática, oferece busca/inserção próximas de $O(1)$.

5.7 Tries

Uma **trie** é uma árvore especializada para armazenar strings (como um dicionário), onde cada nível representa uma letra. Buscar uma palavra de tamanho k custa $O(k)$ — independente de quantas outras palavras existam na trie. É a estrutura usada no pset5 (Speller), o corretor ortográfico do CS50.

5.8 Exercícios — Semana 5

1. Explique a diferença entre uma Fila e uma Pilha, com um exemplo do cotidiano para cada.
2. Qual a vantagem de uma lista ligada sobre um array? E qual a desvantagem?
3. Desenhe (ou descreva em texto) uma Árvore Binária de Busca contendo os números [8, 3, 10, 1, 6, 14].

4. O que é uma “colisão” em uma tabela hash, e como as listas ligadas ajudam a resolvê-la?
5. Por que buscar uma palavra em uma trie não depende do número total de palavras armazenadas?
6. Escreva o struct em C de um nó de lista ligada que armazena uma string.
7. Cite uma vantagem de uma trie sobre uma tabela hash para o caso de um corretor ortográfico.

Semana 6 — Python

6.1 Visão geral

Python: Funções, Argumentos, Valores de Retorno; Variáveis; Expressões Booleanas, Condicionais; Loops. Módulos, Pacotes.

Depois de entender C “por baixo do capô” (memória, tipos, ponteiros), o curso migra para **Python**: uma linguagem de **alto nível**, interpretada (não compilada) e de tipagem dinâmica — muito mais rápida para escrever, ainda que geralmente mais lenta para executar.

6.2 De C para Python: o que muda

Aspecto	C	Python
Execução	Compilado para código de máquina	Interpretado linha a linha
Tipos	Estático (declarado explicitamente)	Dinâmico (inferido automaticamente)
Ponto e vírgula / chaves	Obrigatórios	Não existem — a indentação define blocos
Gerenciamento de memória	Manual (malloc/free)	Automático (coletor de lixo)
Índices de array/lista	Fixo (arrays)	Listas dinâmicas nativas

```
# hello.py
name = input("What's your name? ")
print(f"hello, {name}")
```

6.3 Variáveis e tipos

```
idade = 25          # int
altura = 1.75       # float
nome = "Ana"        # str
ligado = True       # bool
```

Não é preciso declarar o tipo — o Python o infere a partir do valor atribuído.

6.4 Condicionais e expressões booleanas

```
x = int(input("x: "))
y = int(input("y: "))

if x < y:
    print("x is less than y")
elif x > y:
    print("x is greater than y")
else:
    print("x is equal to y")
```

6.5 Loops

```
# while
i = 0
while i < 3:
    print("meow")
    i += 1

# for, iterando sobre um range
for i in range(3):
    print("meow")

# for, iterando diretamente sobre uma lista
for fruta in ["maçã", "banana", "uva"]:
    print(fruta)
```

6.6 Funções, argumentos e valores de retorno

```
def quadrado(n):
    return n * n

for i in range(1, 6):
    print(quadrado(i))
```

- n é o **argumento** (ou parâmetro) da função quadrado.
- `return n * n` devolve o **valor de retorno** para quem chamou a função.

6.7 Módulos e pacotes

Um **módulo** é um arquivo `.py` com funções reutilizáveis; um **pacote** é uma coleção de módulos. O Python vem com uma vasta **biblioteca padrão** e permite instalar pacotes de terceiros via `pip`.

```
import random

numero = random.randint(1, 10) # número aleatório entre 1 e 10 (incluso)
```

O curso também apresenta a biblioteca `cs50` (`pip install cs50`), com funções equivalentes às usadas em C (`get_int`, `get_string`, etc.), facilitando a transição.

```
from cs50 import get_int

idade = get_int("Idade: ")
print(f"Você tem {idade} anos.")
```

6.8 Exercícios — Semana 6

1. Reescreva em Python o programa em C do exercício 1 da Semana 1 (pedir o nome e cumprimentar).
2. Escreva uma função `eh_par(n)` que retorne `True` se `n` for par e `False` caso contrário.
3. Qual a principal diferença entre `for i in range(5)` e `for item in lista`?
4. Explique por que, em Python, a indentação é significativa (afeta o comportamento do programa).
5. Escreva uma função `fatorial(n)` em Python, de forma recursiva.
6. O que é um módulo em Python? Dê um exemplo de um módulo da biblioteca padrão e uma função dele.
7. Compare: cite duas vantagens de Python sobre C, e duas vantagens de C sobre Python.

Semana 7 — SQL

7.1 Visão geral

SQL: Tabelas; Tipos; Instruções; Restrições; Índices; Palavras-chave, Funções; Transações. Condições de corrida (Race Conditions). Ataques de Injeção SQL (SQL Injection).

7.2 Bancos de dados relacionais e tabelas

Um **banco de dados relacional** organiza dados em **tabelas** (linhas = registros, colunas = atributos). **SQL** (Structured Query Language) é a linguagem padrão para criar, consultar e modificar esses dados.

```
CREATE TABLE alunos (  
  id INTEGER,  
  nome TEXT NOT NULL,  
  nota REAL,  
  PRIMARY KEY (id)  
);
```

7.3 Tipos de dados em SQL

Tipo	Uso
INTEGER	Números inteiros
REAL	Números de ponto flutuante
TEXT	Texto (strings)
BLOB	Dados binários (imagens, arquivos)
NUMERIC	Datas, valores monetários, booleanos

7.4 As instruções fundamentais (CRUD)

```
-- CREATE: inserir dado  
INSERT INTO alunos (id, nome, nota) VALUES (1, 'Ana', 9.5);
```

```
-- READ: consultar dados
SELECT nome, nota FROM alunos WHERE nota >= 7 ORDER BY nota DESC;

-- UPDATE: atualizar dado
UPDATE alunos SET nota = 10 WHERE id = 1;

-- DELETE: remover dado
DELETE FROM alunos WHERE nota < 5;
```

7.5 Restrições e índices

- **Restrições (constraints):** regras que garantem a integridade dos dados, como PRIMARY KEY (identificador único), NOT NULL (campo obrigatório), UNIQUE, FOREIGN KEY (referência a outra tabela).
- **Índices:** estruturas auxiliares (parecidas com o índice de um livro) que aceleram buscas em colunas específicas, ao custo de espaço extra e de deixar inserções um pouco mais lentas.

```
CREATE INDEX idx_nome ON alunos (nome);
```

7.6 Funções e palavras-chave úteis

```
SELECT COUNT(*) FROM alunos;
SELECT AVG(nota) FROM alunos;
SELECT * FROM alunos WHERE nome LIKE 'A%'; -- começa com "A"
SELECT * FROM alunos GROUP BY nota;
```

JOIN combina dados de múltiplas tabelas relacionadas:

```
SELECT alunos.nome, turmas.nome_turma
FROM alunos
JOIN turmas ON alunos.turma_id = turmas.id;
```

7.7 Transações e condições de corrida

Uma **transação** agrupa várias operações SQL em uma unidade “tudo ou nada” (*atômica*): ou todas as operações acontecem, ou nenhuma acontece — essencial, por exemplo, para transferências bancárias.

```
BEGIN TRANSACTION;
UPDATE contas SET saldo = saldo - 100 WHERE id = 1;
UPDATE contas SET saldo = saldo + 100 WHERE id = 2;
COMMIT;
```

Uma **condição de corrida (race condition)** ocorre quando dois processos leem e escrevem o mesmo dado “ao mesmo tempo”, gerando um resultado incorreto —

ex.: duas pessoas comprando o último ingresso disponível simultaneamente, e ambas conseguindo, porque cada uma leu “1 disponível” antes da outra atualizar o valor.

7.8 Ataques de SQL Injection

Ocorrem quando entradas do usuário são inseridas diretamente numa query SQL sem tratamento, permitindo que um atacante manipule a lógica da consulta.

```
# VULNERÁVEL – nunca faça isso!  
query = f"SELECT * FROM users WHERE username = '{username}'"  
# se username = "" OR '1'='1", a condição vira sempre verdadeira!
```

Solução: usar **consultas parametrizadas (prepared statements)**, que tratam a entrada do usuário como dado puro, nunca como código SQL:

```
# SEGURO  
db.execute("SELECT * FROM users WHERE username = ?", username)
```

7.9 Exercícios — Semana 7

1. Escreva uma instrução CREATE TABLE para uma tabela livros com id, titulo, autor e ano.
2. Escreva uma consulta SELECT que retorne todos os livros publicados depois de 2010, ordenados por título.
3. O que faz a restrição NOT NULL? E UNIQUE?
4. Explique, com suas palavras, o que é uma condição de corrida, com um exemplo além do citado no texto.
5. Por que uma consulta parametrizada previne SQL Injection, mas concatenar strings não?
6. O que é uma transação em SQL, e por que ela precisa ser “atômica”?
7. Escreva uma consulta usando JOIN para relacionar livros com uma tabela empréstimos.

Semana 8 — HTML, CSS, JavaScript

8.1 Visão geral

Internet: Roteadores; TCP/IP; DNS. HTTP: URLs, GET, POST. HTML: Tags; Atributos. Servidores. Expressões Regulares. CSS: Propriedades; Seletores. Frameworks. JavaScript: Variáveis; Condicionais; Loops. Eventos.

8.2 Como funciona a Internet

- **Roteadores:** dispositivos que encaminham pacotes de dados entre redes até o destino.
- **TCP/IP:** o conjunto de protocolos que permite que computadores se comuniquem. O **IP** endereça cada dispositivo (ex.: 192.168.1.1); o **TCP** garante que os dados cheguem completos e na ordem certa, dividindo mensagens em pacotes.
- **DNS (Domain Name System):** traduz nomes legíveis por humanos (cs50.harvard.edu) em endereços IP.

8.3 HTTP: como o navegador conversa com o servidor

HTTP (HyperText Transfer Protocol) define como clientes (navegadores) e servidores trocam mensagens pela web.

- **URL:** endereço de um recurso, ex.: `https://cs50.harvard.edu/x/weeks/8/`.
- **GET:** solicita dados de um servidor (ex.: carregar uma página). Parâmetros vão na própria URL (`?nome=valor`).
- **POST:** envia dados ao servidor (ex.: enviar um formulário de login), sem expor os dados na URL.

8.4 HTML: estrutura da página

```
<!DOCTYPE html>  
<html lang="pt-br">
```

```

<head>
  <title>Minha Página</title>
</head>
<body>
  <h1>Olá, mundo!</h1>
  <p>Este é um parágrafo.</p>
  <a href="https://cs50.harvard.edu">Link para o CS50</a>
  
</body>
</html>

```

Cada elemento é uma **tag** (<h1>, <p>, <a>), que pode ter **atributos** (href, src, alt) fornecendo informações extras.

8.5 CSS: estilizando a página

CSS (Cascading Style Sheets) controla a aparência visual do HTML através de **seletores** (o que estilizar) e **propriedades** (como estilizar):

```

body {
  font-family: sans-serif;
  background-color: #f0f0f0;
}

h1 {
  color: navy;
  text-align: center;
}

.destaque {          /* seletor de classe */
  font-weight: bold;
}

#titulo-principal { /* seletor de id */
  font-size: 2em;
}

```

Frameworks CSS (como Bootstrap) fornecem componentes e classes prontas para acelerar o desenvolvimento, evitando reescrever estilos comuns do zero.

8.6 Expressões regulares (Regex)

Padrões usados para buscar, validar ou extrair texto:

```
^[\w.+-]+@[ \w-]+\.[a-zA-Z]{2,}$
```

Esse padrão valida um formato básico de e-mail: caracteres válidos antes do @, um domínio, e uma extensão de pelo menos 2 letras.

8.7 JavaScript: interatividade no navegador

Diferente de HTML/CSS (estrutura e estilo), **JavaScript** é uma linguagem de programação completa que roda no navegador, permitindo reagir a **eventos** e alterar a página dinamicamente (manipulando o DOM).

```
<button id="botao">Clique aqui</button>
<script>
    let contador = 0;    // variável

    document.querySelector("#botao").addEventListener("click", function () {
        contador++;
        if (contador === 1) {
            alert("Você clicou " + contador + " vez!");
        } else {
            alert("Você clicou " + contador + " vezes!");
        }
    });
</script>

// loop em JavaScript
for (let i = 0; i < 5; i++) {
    console.log(i);
}
```

8.8 Exercícios — Semana 8

1. Explique, em uma frase cada, o papel do IP, do TCP e do DNS.
2. Qual a diferença entre uma requisição GET e uma POST? Dê um exemplo de uso de cada uma.
3. Escreva uma página HTML simples com um título, um parágrafo e uma lista não ordenada () de 3 itens.
4. Escreva um seletor CSS que deixe todos os links (<a>) verdes e sem sublinhado.
5. Qual a diferença entre um seletor de classe (.minha-classe) e um seletor de id (#meu-id) em CSS?
6. Escreva uma função em JavaScript que receba um número e retorne "par" ou "ímpar".
7. O que é o DOM, e por que o JavaScript precisa dele para alterar uma página já carregada?

Semana 9 — Flask

9.1 Visão geral

Flask. Rotas (Route). Decoradores. Requisições, Respostas. Sessões. Cookies.

Flask é um *microframework* em Python para construir aplicações web — o lado do **servidor (back-end)**, que recebe requisições HTTP e devolve respostas (muitas vezes, páginas HTML geradas dinamicamente).

9.2 Estrutura mínima de uma aplicação Flask

```
from flask import Flask, render_template, request

app = Flask(__name__)

@app.route("/")
def index():
    return render_template("index.html")
```

- `@app.route("/")` é um **decorador**: associa a função `index()` à **rota** (URL) `/`. Quando alguém acessa esse endereço, o Flask chama essa função.
- `render_template` usa o motor de templates **Jinja** para gerar HTML dinamicamente a partir de um arquivo `.html` na pasta `templates/`.

9.3 Rotas com múltiplos métodos e formulários

```
@app.route("/greet", methods=["GET", "POST"])
def greet():
    if request.method == "POST":
        nome = request.form.get("nome", "mundo")
        return render_template("cumprimento.html", nome=nome)
    return render_template("formulario.html")
```

- `request.method` diz se a requisição foi GET ou POST.

- `request.form` contém os dados enviados por um formulário HTML (POST); `request.args` contém dados enviados por GET (na própria URL, ex.: `?nome=Ana`).

Template Jinja correspondente (`templates/cumprimento.html`):

```
<!DOCTYPE html>
<html>
  <body>
    <h1>Olá, {{ nome }}!</h1>
  </body>
</html>
```

`{{ nome }}` insere dinamicamente o valor da variável Python `nome` no HTML.

9.4 Sessões e cookies

HTTP é um protocolo sem estado (*stateless*) — cada requisição é independente, o servidor não “lembra” do usuário entre uma e outra por padrão. Para contornar isso:

- **Cookies:** pequenos pedaços de dados que o servidor pede ao navegador para guardar e reenviar em cada requisição futura (ex.: um identificador de sessão).
- **Sessões (sessions):** dados associados a um usuário específico, guardados no servidor (Flask usa um cookie para identificar *qual* sessão pertence a cada navegador).

```
from flask import Flask, session
from flask_session import Session

app = Flask(__name__)
app.config["SESSION_PERMANENT"] = False
app.config["SESSION_TYPE"] = "filesystem"
Session(app)

@app.route("/login", methods=["POST"])
def login():
    session["usuario_id"] = 1  # guarda o id do usuário na sessão
    return "Login realizado!"

@app.route("/perfil")
def perfil():
    if "usuario_id" not in session:
        return "Você precisa fazer login primeiro."
    return f"Bem-vindo, usuário {session['usuario_id']}!"
```

É assim que sites “lembram” que você está logado ao navegar entre páginas.

9.5 Exercícios — Semana 9

1. O que faz o decorador `@app.route("/sobre")` em uma aplicação Flask?

2. Qual a diferença entre `request.form` e `request.args`?
3. Escreva uma rota Flask `"/soma"` que receba dois números via GET (`?a=2&b=3`) e retorne a soma como texto.
4. Por que dizemos que o HTTP é um protocolo “sem estado” (*stateless*)?
5. Explique, com suas palavras, a diferença entre um cookie e uma sessão.
6. O que é o Jinja, e para que serve a sintaxe `{{ variavel }}` em um template?
7. Por que é importante nunca guardar a senha do usuário diretamente na sessão ou em um cookie?

Semana 10 — The End

10.1 Visão geral

A última semana do CS50x é, nas palavras do próprio site do curso, dedicada a “**Fun. Games.**” (diversão e jogos) — uma aula de encerramento mais leve, seguida de **seminários** sobre tópicos avançados escolhidos pelos alunos, e com foco total na conclusão do **Projeto Final (Final Project)**.

10.2 O Projeto Final

O Projeto Final é a culminação do curso: um programa da sua própria autoria, usando uma ou mais das tecnologias aprendidas ao longo das semanas (C, Python, SQL, HTML/CSS/JavaScript, Flask), que resolva um problema de sua escolha. Ele deve:

- Ser **maior e mais complexo** do que qualquer pset (problem set) do curso;
- Ser **original** — não uma cópia de um projeto ou tutorial existente;
- Incluir um vídeo curto de demonstração e um arquivo README.md explicando o projeto e como ele foi construído.

10.3 Recapitulando a jornada

Semana	Tema	Ideia central
0	Scratch	Pensamento computacional e abstração
1	C	Sintaxe, tipos, correção/design/estilo
2	Arrays	Compilação, arrays, strings, criptografia
3	Algoritmos	Busca, ordenação, Big O, recursão
4	Memória	Ponteiros, stack/heap, alocação dinâmica
5	Estruturas de Dados	TADs, listas ligadas, árvores, hash, tries
6	Python	Alto nível, tipagem dinâmica, produtividade
7	SQL	Bancos de dados relacionais e segurança
8	HTML/CSS/JS	A web: estrutura, estilo e interatividade
9	Flask	Aplicações web no back-end com Python
10	The End	Projeto Final e próximos passos

10.4 Seminários e além do CS50x

O curso incentiva a explorar tópicos adicionais por meio de **seminários** (aulas curtas e opcionais sobre temas como Git, Unity, desenvolvimento mobile, entre outros) e aponta para cursos derivados — **CS50 Python**, **CS50 SQL**, **CS50 Web Programming**, **CS50 AI**, **CS50 Cybersecurity** — para quem quiser se aprofundar em uma trilha específica após concluir o CS50x.

10.5 Exercícios — Semana 10

1. Escreva, em algumas frases, uma ideia de Projeto Final que combine ao menos duas tecnologias vistas no curso (ex.: Python + SQL, ou Flask + JavaScript).
 2. Liste, para o seu Projeto Final, quais problemas de semanas anteriores (arrays, ponteiros, estruturas de dados, etc.) você provavelmente vai reencontrar.
 3. Releia a tabela da seção 10.3 e, para cada semana, escreva de memória (sem consultar o resto da apostila) uma palavra-chave que resuma o tema.
 4. Qual das linguagens/ferramentas do curso (C, Python, SQL, HTML/CSS/JS, Flask) mais despertou seu interesse, e por quê?
 5. Depois do CS50x, qual trilha de continuação (Python, SQL, Web, AI, Cybersecurity) faz mais sentido para os seus objetivos, e por quê?
-

Referências

Todo o conteúdo desta apostila foi organizado a partir da página oficial do curso:

- CS50x 2026 — Weeks: <https://cs50.harvard.edu/x/weeks/>
- Semana 0 (Scratch): <https://cs50.harvard.edu/x/weeks/0/>
- Semana 1 (C): <https://cs50.harvard.edu/x/weeks/1/>
- Semana 2 (Arrays): <https://cs50.harvard.edu/x/weeks/2/>
- Semana 3 (Algoritmos): <https://cs50.harvard.edu/x/weeks/3/>
- Semana 4 (Memória): <https://cs50.harvard.edu/x/weeks/4/>
- Semana 5 (Estruturas de Dados): <https://cs50.harvard.edu/x/weeks/5/>
- Semana 6 (Python): <https://cs50.harvard.edu/x/weeks/6/>
- Semana 7 (SQL): <https://cs50.harvard.edu/x/weeks/7/>
- Semana 8 (HTML, CSS, JavaScript): <https://cs50.harvard.edu/x/weeks/8/>
- Semana 9 (Flask): <https://cs50.harvard.edu/x/weeks/9/>
- Semana 10 (The End): <https://cs50.harvard.edu/x/weeks/10/>

CS50 é o curso “Introduction to Computer Science” de Harvard, ministrado por David J. Malan. Esta apostila é um material de estudo complementar, não afiliado oficialmente ao CS50/Harvard.